AI4RO2 Assignment D3-V4

Warehouse Robotics - Capacity constrained transport

Luca Tassoni

May 2026

1 Introduction

The goal of this project is to model a package delivery scenario using PDDL (Planning Domain Definition Language), where one or more robots must optimize their routes while respecting capacity constraints. As required by the assignment, the project presents two distinct modeling approaches:
-In the classical PDDL version, capacity constraints are directly modeled within the domain definition. Three problem instances are provided to demonstrate how robot capacity and package characteristics influence route planning decision. For each problem instance, the corresponding valid plan is presented to illustrate the optimization process.
-In the PDDL+ version, processes are introduced to model the temporal aspects of loading and unloading operations. Additionally, an event is defined to represent overload violations when capacity constraints are not respected. Similarly to the classical approach, three problem instances are provided with their respective valid plans, demonstrating the complex interactions between temporal dynamics and capacity constraints.

2 Q1- Basic PDDL Model

This section illustrates the modeling choices adopted to correctly represent the proposed scenario in classic PDDL.

2.1 Modeling Choices

The model presented envisions a warehouse environment where one or multiple robots operate autonomously. The warehouse is organized into distinct rooms, each can serve two purposes: they may contain packages waiting for delivery, or they may function as delivery stations where packages must be transported. Robots must navigate efficiently through this environment, manage their cargo capacity, and execute delivery tasks by batching packages from their origin rooms and transporting them to designated delivery stations. Since a primary objective of this project is to demonstrate how the planner makes intelligent

decisions regarding batching—grouping multiple packages into single delivery trips—several assumptions are made to isolate and highlight this planning aspect:

- The robots have access to all warehouse rooms.
- All locations are fully connected, enabling free robot movement throughout the warehouse.
- The warehouse is discretized into rooms; once a robot enters a room containing a package, it can be loaded without considering the precise spatial coordinate.
- Multiple robots and packages may occupy the same room without physical constraints.
- For the reason mentioned before, collisions between robots are not modeled.
- Low-level manipulation details, such as how robots grasp, manipulate, and move packages, are not taken into account.
- Robot capacity constraints include weight and size; in particular, the second one is modeled as a single-dimension property rather than real-world three-dimensional space.

2.2 Domain definition

The following section presents the complete PDDL domain implementation. This includes the formal definition of types, predicates, and actions.

2.2.1 Types

PDDL types are used to represent objects of a given type or trivially classes. The hierarchy to model our case-scenario is simple and presents only three main classes:

- **Robot**: this type represents the robots that operate in the warehouse. The various instances of this class are characterized by a distinct carrying capacity.
- **Package**: this type represents objects to be delivered. Objects of this class have associated weight and size.
- **Location**: this type represents rooms within the warehouse. Locations serve as pickup points for packages, delivery stations, or transit nodes through which robots navigate.

2.2.2 Predicates

The predicates in PDDL are logic sentences that describe the property of the objects (class instances) within the world that we are modeling. The set of predicates that are true defines the current states of the world.

- **robot-at**: specifies the current location of a robot within the warehouse.
- **package-at**: specifies the current location of a package.
- **loaded**: indicates that a package has been loaded into a specific robot.
- **delivery-station**: marks locations that serve as delivery destinations.

2.2.3 Functions

The functions in PDDL are tools for representing and manipulating numeric values. In order to fully describe the capacity constraints of the robots and to enable the planner to make informed batching decisions, several numeric functions are introduced. These variables track: the weight and size of the packages, the maximum-weight and size capacity of the robots, and, also their current load. Another function called total-cost will be discussed later.

2.2.4 Actions

An action in PDDL modifies the state of the world by setting to true or false the values of predicates or by updating the value of numeric functions. In order for the planner to choose an action, some preconditions should be verified. In the domain presented, the planner has to take decisions such as moving the robot around the warehouse, loading packages, and delivering them to the station. Indeed, the main actions are the following:

- **Move**: this action makes one robot move from a location to another.
- **Load**: this action allows one robot to load a package from a location, when they occupy the same room and only if the package respects the capacity constraints. As effect, this action increases the values of the current weight and size of the robot.
- **Unload**: this action allows one robot to unload a package in a location. As effect, this action decreases the values of the current weight and size of the robot.

The first version of the project had only these three actions, but after a first test with a simple problem that requires multiple delivery trips, an undesired behavior occurred.

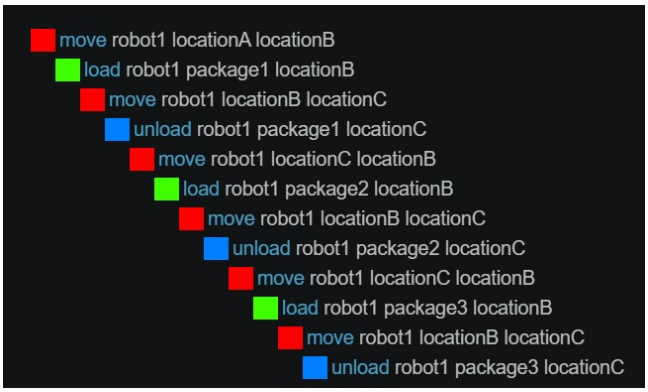

Figure 1: Single package trips

The plan presented above was generated by the planner `BFWS-ff-parser-version`. As illustrated in the figure, this planner returns the first feasible plan discovered during search. Consequently, the robot adopts a suboptimal strategy: delivering one package per trip, which fails to exploit the potential for package batching and thus does not exhibit the desired behavior. To address this limitation, we modified the domain to encourage batching. Specifically, we imposed a precondition on the **move** action: robots may only navigate between locations when they carry no loaded packages. Additionally, we introduced a new action called **delivery-trip**, which forces the robot to load packages greedily from its current location until it reaches its capacity. This design ensures that the planner must batch multiple packages before delivering them. To implement the **delivery-trip** action effectively, we introduced a precondition using the `forall` quantifier to ensure all packages in the robot's current location are evaluated for loading. However, this approach reveals a critical limitation: many PDDL planners provide incomplete support for `forall` constructs; in particular, they silently ignore the quantifier condition by treating it as trivially true, without raising errors or warnings. We investigated alternative formulations avoiding `forall` and `exists` quantifiers to improve planner compatibility. One approach introduced a `to-check` predicate for each package and a `remaining-check` function that decrements as packages are evaluated. However, this strategy required resetting `remaining-check` after each package loading and after each new delivery trip, introducing substantial complexity without proportional benefit to the domain logic. Given this trade-off, we opted to retain the simpler formulation using `forall`, accepting the constraint that valid planning requires a planner with support for quantifiers, such as ENHSP.

2.3 Problems and plans

This section presents three problem instances designed to demonstrate how the planner makes strategic decisions. For each instance, we provide the corresponding valid plan generated, analyzing how capacity constraints influence the

batching decisions and overall delivery efficiency.

2.3.1 Problem-1

Problem 1 presents a simple warehouse scenario with a single robot, three packages, and three locations. Initially, the robot is positioned at `locationA`, while all three packages are located at `locationB`. The delivery destination is `locationC`. Each package has distinct weight and size properties, while the robot has a maximum capacity of weight 5 and size 3. Since the sum of all packages weights and sizes exceeds the robot's capacity, the planner is forced to partition packages across multiple trips. The plan provided by the ENHSP planner is the following.

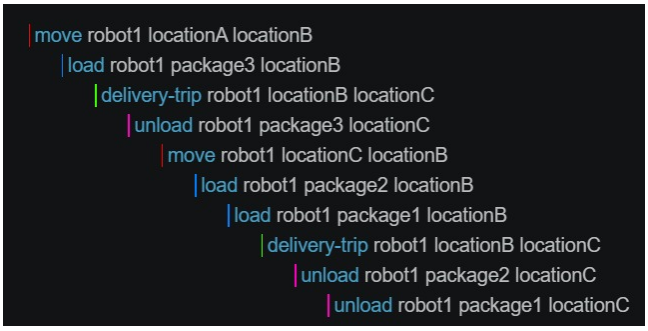

Figure 2: problem1 plan ENHSP

2.3.2 Problem-2

Problem 2 introduces a more permissive capacity environment with weight limit 10 and size limit 5. The combined package specifications fit exactly within the robot's capacity, allowing the planner to transport all three packages in a single delivery trip. This problem demonstrates the planner's ability to recognize when maximal batching is possible and optimal. Figure 3 presents the resulting plan.

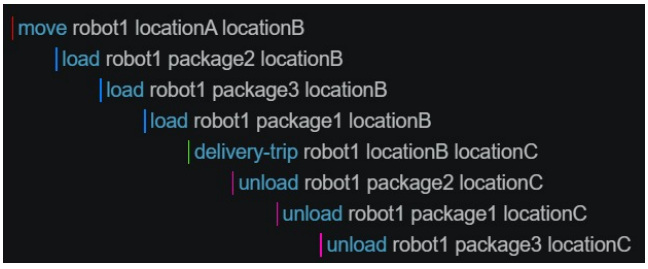

Figure 3: problem2 plan ENHSP

2.3.3 Problem-3

Problem 3 introduces a more complex scenario with four packages and the same robot capacity constraints as Problem 2. This problem is designed to illustrate a critical limitation of the domain design: the **delivery-trip** action enforces greedy package loading, indeed, the robot loads packages sequentially until capacity is reached, but does not guarantee optimal batching.

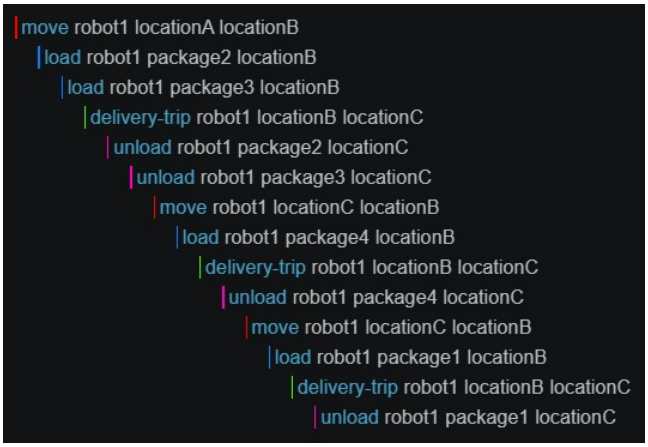

Figure 4: problem3 plan ENHSP

Figure 4 clearly demonstrates how the planner executes a total of 3 delivery trips, despite an optimal solution requiring only 2 trips. This behavior stems from the heuristic nature of ENHSP, which is designed for efficiency and scalability rather than optimality. Once a feasible solution is found, the planner commits to it without exploring alternatives. The discussion of plan optimality and strategies for improvement is presented in Section 2.4.

2.4 Optimality

The observation that the greedy loading strategy produces suboptimal plans motivated an investigation into alternative approaches that could guide the planner toward truly optimal solutions. Multiple strategies were explored, each revealing different limitations in current planning technology.
We tested ENHSP's `-planner opt-hrmax` option designed for exhaustive search. However, it failed to discover superior alternatives, indicating that ENHSP's heuristic-guided approach remains limited. Furthermore, we introduced a `total-cost` function with metric minimization and `:action-cost` requirements. While conceptually sound, ENHSP's heuristic search do not explore sufficient action combinations to find true global minima, it optimize locally without knowledge of global optima. Therefore, we tested it with other planner like `Fast Downward`, which handles metrics more robust and removed the `forall` quantifier for com-

patibility. However, this planner encountered issues with complex numeric calculations, such as cumulative weight sums during loading, producing errors during the plan evaluation.
A theoretically viable solution would encode weight and size as discrete states rather than continuous values, allowing the planner to operate correctly. However, this transformation would reduce the real-world bin-packing problem to abstract state search, which is a less realistic delivery scenario.

3 Q2- PDDL+ Model

The basic PDDL domain of section 2 model the world with instantaneous actions. To introduce temporal realism and runtime constraint monitoring, this section provide an extended formulation using PDDL+.

3.1 Domain definition

3.1.1 Modeling Choices

The PDDL+ domain extends the classical model with temporal dynamics while maintaining its core structure. Loading and unloading are now modeled as temporal processes rather than instantaneous actions, reflecting the time required for package manipulation; indeed, the time required during those processes is directly proportional to the weight of the package. This introduces temporal reasoning into the planner's decision-making.
The move actions remain a simple instantaneous actions, not durative. This design prioritizes capacity constraint modeling by avoiding unnecessary temporal complexity in navigation, keeping the focus on how capacity influences batching decisions.

3.1.2 PDDL+ Domain definition

The following section presents the modifications applied to the classical PDDL domain to convert it into a temporal PDDL+ model

3.1.3 Predicates

We added three new predicates, necessary to correctly model the world taking in consideration the time.

- **loading**: indicates that a robot is loading a package.
- **unloading**: same functionality of the previous predicate but for the unloading process.
- **busy**: specify if a robot is busy in some loading or unloading process. This variable prevents the robot from moving while loading or unloading processes.

3.1.4 Functions

We added some numeric function to implement correctly the new model.

- **loading-temp-weight**: a temporary variable to update without error the current weight of the robot;
- **loading-temp-size**: a temporary variable to update without error the current size of the robot;
- **loading-progress**: a variable that tracks the progress of the loading process;
- **unloading-progress**: a variable that tracks the progress of the unloading process;

3.1.5 Actions

- **start-loading**: it starts the loading process setting the loading predicate to true and resetting the numeric functions involved.
- **start-unloading**: it starts the unloading process setting the unloading predicate to true and resetting the numeric functions involved.

3.1.6 Processes

- **loading-process**: this process incrementally advances the loading progress variable over time. The duration is proportional to the weight of the package involved. Instead of directly modifying the current weight and size of the robot, it increases the temporary variables, which will trigger the overload event;
- **unloading-process**: this process acts in the same way for the unloading process, but without using temporary variables.

3.1.7 Events

- **finish-loading**: this event is triggered when loading-progress is greater than 1. This means that the loading process has terminated correctly and the current capacity of the robot can be updated safely.
- **finish-unloading**: this event is triggered when loading-progress is greater than 1, allowing the robot to actually deliver a package in a location.
- **Overload**: this event is triggered during the loading process when one of the two capacity constraint is exceeded.

3.2 Problems and plans

Equally to the other domain, three problems are presented to demonstrate the efficiency of the planner.

3.2.1 Problem-1

The first problem is analogous to the problem-2, shown in subsection 2.3.2. The packages do not exceed the robot's capacity constraints, requiring only a single delivery trip for the robot to complete its task. The resulting plan has a total duration of exactly 18 seconds, which equals twice the sum of all package weights. This relationship reflects the domain design where loading and unloading duration is directly proportional to package weight.

```
Found Plan:
0: (start-loading robot1 package3 warehouse)
0: -----waiting---- [4.0]
4.0: (start-loading robot1 package1 warehouse)
4.0: -----waiting---- [6.0]
6.0: (start-loading robot1 package2 warehouse)
6.0: -----waiting---- [9.0]
9.0: (delivery-trip robot1 warehouse delivery1)
9.0: (start-unloading robot1 package1 delivery1)
9.0: -----waiting---- [11.0]
11.0: (start-unloading robot1 package2 delivery1)
11.0: -----waiting---- [14.0]
14.0: (start-unloading robot1 package3 delivery1)
14.0: -----waiting---- [18.0]
```

Figure 5: problem1 plan PDDL+ ENHSP

3.2.2 Problem-2

This problem demonstrates how the PDDL+ model enforces capacity constraints during temporal execution, consistent with the classical PDDL behavior observed in Problem Instance 3 (Subsection 2.3.3).

```
Found Plan:
0: (start-loading robot1 p3 warehouse)
0: -----waiting---- [4.0]
4.0: (start-loading robot1 p1 warehouse)
4.0: -----waiting---- [8.0]
8.0: (delivery-trip robot1 warehouse delivery1)
8.0: (start-unloading robot1 p3 delivery1)
8.0: -----waiting---- [12.0]
12.0: (start-unloading robot1 p1 delivery1)
12.0: -----waiting---- [16.0]
16.0: (move robot1 delivery1 warehouse)
16.0: (start-loading robot1 p2 warehouse)
16.0: -----waiting---- [21.0]
21.0: (delivery-trip robot1 warehouse delivery1)
21.0: (start-unloading robot1 p2 delivery1)
21.0: -----waiting---- [26.0]
```

Figure 6: problem1 plan PDDL+ ENHSP

3.2.3 Problem-3

To further complicate the scenario, this problem introduces two robots with different capacity constraints and four packages distributed across two warehouse zones. The increased complexity of coordinating multiple robots and packages

from different locations revealed a limitation: the PDDL+ planner provided by the VS Code extension failed to generate a valid plan, likely due to the problem's complexity exceeding the online planner's capabilities. To resolve this, we installed ENHSP locally and re-ran the problem. The local planner successfully generated a coherent and correct plan, demonstrating that the domain formulation is sound and that multi-robot coordination is properly modeled in PDDL+.

```
Found Plan:
0: (move r2 dispatch-1 zone-B)
0: (start-loading r1 p2 zone-A)
0: (move r2 zone-B zone-A)
0: -----waiting---- [5.0]
5.0: (move r2 zone-A zone-B)
5.0: (delivery-trip r1 zone-A dispatch-1)
5.0: (start-unloading r1 p2 dispatch-1)
5.0: -----waiting---- [10.0]
10.0: (move r2 zone-B dispatch-1)
10.0: (move r1 dispatch-1 zone-B)
10.0: (start-loading r1 p4 zone-B)
10.0: -----waiting---- [18.0]
18.0: (delivery-trip r1 zone-B dispatch-1)
18.0: (start-unloading r1 p4 dispatch-1)
18.0: -----waiting---- [26.0]
26.0: (move r1 dispatch-1 zone-A)
26.0: (start-loading r1 p1 zone-A)
26.0: -----waiting---- [32.0]
32.0: (delivery-trip r1 zone-A dispatch-1)
32.0: (start-unloading r1 p1 dispatch-1)
32.0: -----waiting---- [38.0]
38.0: (move r2 dispatch-1 zone-B)
38.0: (start-loading r2 p3 zone-B)
38.0: -----waiting---- [50.0]
50.0: (delivery-trip r2 zone-B dispatch-1)
50.0: (start-loading r1 p2 dispatch-1)
50.0: -----waiting---- [55.0]
55.0: (start-unloading r1 p2 dispatch-1)
55.0: -----waiting---- [60.0]
60.0: (move r1 dispatch-1 zone-A)
60.0: (start-unloading r2 p3 dispatch-1)
60.0: -----waiting---- [72.0]
```

Figure 7: problem3 plan PDDL+ ENHSP

It can be observed that, although the domain does not impose any constraints on parallel execution between multiple robots, the planner tends to schedule their tasks sequentially rather than exploiting the available parallelism, resulting again in a suboptimal solution.